



INTERNATIONAL INSTITUTE OF
INFORMATION TECHNOLOGY

H Y D E R A B A D

AIML B26 - PGCP

CodeGen

*An Agentic AI System for Text-to-Python Generation, Java-to-Python Conversion, and
Code Documentation*

Group - 19

Gopal Dhaked

Anup Guduru

Sushmita Minukuri

Venkata Sandhya Nerella

Project Mentor: Pawan & Nayan

Project Supervisor: Prof. Anil

May 31, 2026

1. Introduction

2. Text-to-Python Code Generation (Task 1)

2.1. Literature Review

2.2. Proposed Approach

2.2.1. Dataset

2.2.2. Model

2.2.3. Training, Evaluation, and Deployment

2.3. Applicability in the Real World

3. Java-to-Python Code Conversion (Task 2)

3.1. Literature Review

3.2. Dataset

3.3. Proposed Approach

3.4. Applicability in the Real World

4. Code Documentation and Explanation (Task 3)

4.1. Literature Review

4.2. Dataset

4.3. Proposed Approach

4.4. Applicability in the Real World

5. Agentic AI System Design

5.1. Streamlit User Interface

5.2. LangGraph Orchestration

5.3. FastAPI Services and Tool Layer

5.4. Validation and Response Generation

6. Challenges

7. Project Timeline

8. References

1. Introduction

This document presents the project proposal for CodeGen, an Agentic AI system designed to solve three software-engineering tasks: generating Python code from natural-language instructions, converting Java programs into Python, and producing documentation and explanations for source code.

The overall objective of the project is to study open-source code language models, evaluate their baseline performance, fine-tune them for selected tasks, and integrate them into a single user-facing prototype using Google Colab, LangGraph, FastAPI, Streamlit, and PySpark.

2. Text-to-Python Code Generation (Task 1)

The objective of this task is to convert natural-language user instructions into correct, readable, and executable Python code. This task is valuable because it supports beginner programmers, reduces repetitive development effort, and improves productivity in rapid prototyping scenarios.

2.1. Literature Review

Recent progress in code generation has been driven by transformer-based code models and benchmark datasets that evaluate natural-language-to-code synthesis. Research in this area shows that execution-based correctness, semantic alignment, and structure-aware metrics are more reliable than plain text-overlap measures when evaluating generated code.

2.2. Proposed Approach

2.2.1. Dataset

For this task, CodeParrot will be used as a supplemental Python-domain corpus, while instruction-to-code benchmarks such as MBPP and APPS will be used for supervised evaluation and task alignment. This combination is more appropriate than relying only on a monolingual Python corpus, because Text-to-Python generation requires natural-language prompts paired with executable solutions.

2.2.2. Model

The project will start with Salesforce CodeGen-350M-multi as the baseline model because it is lightweight enough for initial experimentation and supports multilingual code generation. Based on results and resource availability, the team may later compare the baseline with stronger open-source alternatives such as Qwen Coder or DeepSeek Coder.

2.2.3. Training, Evaluation, and Deployment

The work will begin by measuring the baseline performance of the pretrained model. After that, the model will be adapted using parameter-efficient fine-tuning approaches such as LoRA or QLoRA so that experimentation remains feasible in Google Colab. Performance will be measured using execution

accuracy, CodeBLEU, syntax validity, and CodeBERT-based semantic evaluation. The selected model will later be exposed through a FastAPI service for integration into the final agentic system.

2.3. Applicability in the Real World

This task can support educational coding assistants, developer productivity tools, rapid prototyping systems, and enterprise automation platforms where users need Python solutions from plain-English requirements.

3. Java-to-Python Code Conversion (Task 2)

The objective of this task is to translate Java programs into equivalent Python programs while preserving the logic, structure, and functional behavior of the source program. This task is especially relevant for software modernization, academic learning, and migration from legacy systems.

3.1. Literature Review

Program translation requires more than simple token replacement. It must preserve semantics, control flow, and language-specific conventions. Public resources such as AVATAR and XLCoST show that parallel multilingual datasets provide a stronger academic foundation for code translation than monolingual datasets alone.

3.2. Dataset

This task will use publicly available Java-Python parallel datasets such as AVATAR and XLCoST. During preprocessing, the data will be cleaned and filtered so that the training set contains representative examples with clear source-target mappings and, wherever possible, runnable code pairs.

3.3. Proposed Approach

The baseline CodeGen model will first be evaluated directly on Java-to-Python conversion prompts. The model will then be adapted using paired Java-Python examples. Prompt templates will explicitly specify the source language, target language, and output expectations. Performance will be evaluated using CodeBLEU, syntactic correctness, and execution-based validation wherever testable examples are available.

3.4. Applicability in the Real World

This task is useful for legacy application migration, multi-language code maintenance, developer training, and cross-language understanding of software logic.

4. Code Documentation and Explanation (Task 3)

The objective of this task is to generate natural-language explanations and documentation for input code. The system should be able to summarize functions, explain the logic of code snippets, and support readable docstring generation for better maintainability.

4.1. Literature Review

Code documentation is closely related to code summarization and code-to-text generation. Prior work using paired code-and-description corpora has shown that code-specialized language models can generate useful summaries, explanations, and function-level documentation when trained on aligned code-text datasets.

4.2. Dataset

For this task, a public code-text dataset such as CoDesc or CodeSearchNet will be used. These datasets provide a stronger academic basis for the proposal than an unspecified internal corpus and align well with the goal of generating explanations and documentation from source code.

4.3. Proposed Approach

The baseline CodeGen model will first be evaluated on explanation prompts without additional task-specific tuning. It will then be adapted using code-description datasets to improve summary quality, clarity, and faithfulness to the source code. Performance will be measured through semantic similarity, readability, and manual review of explanation quality on representative examples.

4.4. Applicability in the Real World

This task can support software teams in project documentation, developer onboarding, code review, and faster understanding of poorly documented or legacy codebases.

5. Agentic AI System Design

The final deliverable of this project will be an Agentic AI system that intelligently routes user requests to the correct specialized component and returns validated output through a unified interface.

5.1. Streamlit User Interface

A Streamlit-based interface will be developed so that users can submit natural-language prompts, paste Java code for conversion, or provide source code for explanation. The interface will present clean inputs and outputs for all three use cases.

5.2. LangGraph Orchestration

LangGraph will be used as the orchestration layer to analyze user intent and route each request to the appropriate sub-agent: Text-to-Python, Java-to-Python, or Code Documentation. This will improve modularity and make the final system easier to extend.

5.3. FastAPI Services and Tool Layer

Each specialized model will be exposed through a FastAPI endpoint. These endpoints will act as callable tools for the orchestrator and will allow independent experimentation, modular deployment, and easier system integration.

5.4. Validation and Response Generation

Before returning the final response, the system will perform lightweight validation such as syntax checks, formatting cleanup, and consistency checks. For runnable cases, execution-based validation will be applied wherever feasible to improve reliability.

6. Challenges

- Selecting and cleaning datasets that are strongly aligned with each task, especially for Java-to-Python conversion.
- Managing compute constraints in Google Colab while working with large models and large-scale code corpora.
- Ensuring that generated or translated code is functionally correct and not merely syntactically valid.
- Measuring explanation quality for documentation tasks, where fluent output may still be incomplete or inaccurate.
- Maintaining reliable orchestration when multiple task-specific agents are combined into a single workflow.

7. Project Timeline

The project will be executed in phased milestones as shown below.

Phase	Activities	Target Date
Phase 1	Finalize the baseline model, confirm datasets for all three tasks, define evaluation metrics, and prepare the preprocessing strategy.	June 7
Phase 2	Measure baseline performance of CodeGen-350M-multi and establish comparison criteria for all task modules.	June 12

Phase 3	Perform task-specific adaptation for Text-to-Python and Code Documentation, and prepare curated data for Java-to-Python conversion.	June 27
Phase 4	Build the Streamlit UI, FastAPI services, and LangGraph orchestration layer; integrate the full workflow and prepare the final demo.	July 12

8. References

1. Nijkamp, E., et al. (2022). CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis. <https://arxiv.org/abs/2203.13474>
2. Feng, Z., et al. (2020). CodeBERT: A Pre-Trained Model for Programming and Natural Languages. <https://aclanthology.org/2020.findings-emnlp.139/>
3. Ren, S., et al. (2020). CodeBLEU: a Method for Automatic Evaluation of Code Synthesis. <https://arxiv.org/abs/2009.10297>
4. Austin, J., et al. (2021). Program Synthesis with Large Language Models. <https://arxiv.org/abs/2108.07732>
5. Hugging Face. codeparrot/codeparrot-clean dataset. <https://huggingface.co/datasets/codeparrot/codeparrot-clean>
6. Ahmad, W. U., et al. (2021). AVATAR: A Parallel Corpus for Java-Python Program Translation. <https://arxiv.org/abs/2108.11590>
7. Hasan, M., et al. (2021). CoDesc: A Large Code-Description Parallel Dataset. <https://aclanthology.org/2021.findings-acl.18/>
8. LangChain. LangGraph documentation. <https://docs.langchain.com/langgraph>
9. Streamlit official documentation.